Qwen

من الصفر إلى النجاح: دليل برمجي متكامل لتطوير أداة ESP لـ PUBG Mobile على LDPlayer 14

البيئة المستهدفة: عمارة LDPlayer 14 والتحديات المترتبة عليها

إن تطوير أداة مساعدة مثل "الإدراك الخارق" للعبة PUBG Mobile يتطلب فهمًا عميقًا ليس فقط للعبة نفسها، بل أيضًا للبيئة التي تعمل فيها، وهي في هذه الحالة محاكي Android LDPlayer 14. يشكل فهم العمارة الداخلية للمحاكي والاختلافات بينه وبين المحاكيات الأخرى حجر الزاوية الذي يُبنى عليه أي حل برمجي ناجح ومستدام. إن الفشل في إدراك هذه الاختلافات قد يؤدي إلى جهود برمجية باهظة الثمن تتطلب إعادة تصميم من الأساس بعد كل تحديث طفيف للعبة أو المحاكي. تكشف المواد المتاحة عن نقطة حرجة ومفصلية: عدم توافق الحلول البرمجية بين المحاكيات المختلفة، وهو ما يفرض على المطورين اتباع نهج متخصص وموجه نحو كل بيئة على حدة.

تتميز LDPlayer 14 بخطوة تكنولوجية كبيرة متمثلة في اعتماده على أساس Android 14 [37] [38]. هذا التغيير الجذري مقارنة بالإصدارات السابقة مثل LDPlayer 9 التي كانت تعتمد على Android 9/12، يفتح الباب أمام تشغيل الألعاب الجديدة ذات المتطلبات العالية لواجهة برمجة التطبيقات للتطبيقات (SDK) [45] [47]. ومع ذلك، فإن هذا التحديث لا يقتصر على مجرد التوافق؛ فهو يعيد تشكيل الطبقات الأساسية للتفاعل مع النظام. أحد أبرز التطورات هو دعمه الأصلي لوحدة التحكم في Hyper-V (WHPX) في نظام التشغيل Windows [37] [47]. هذا يعني أن LDPlayer 14 يمكنه الآن التنافس بفعالية أكبر مع تقنيات الافتراضية الأخرى مثل Windows Sandbox و WSL2، يمثل هذا الدعم بالنسبة لمطور أداة ESP، دون الحاجة إلى تعطيل ميزات النظام الأساسية [45]. الجديد إمكانات غير مستغلة تمامًا حتى الآن، حيث يمكن أن يوفر طرقًا أكثر مباشرة وأقل كشفًا للوصول إلى ذاكرة المحاكي الافتراضي. على سبيل المثال، يمكن استكشاف استخدام واجهة BHYPER Platform API (WHPX) لقراءة ذاكرة العملية مباشرة من مستوى النفق، مما يتجاوز نظام التشغيل الضيف بالكامل ويقدم طريقة قوية للتغلب على حواجز الذاكرة الخارجية [53].

على الرغم من هذه التطورات، لا تزال بعض المتطلبات الأساسية ضرورية لتحقيق الأداء الأمثل. تتطلب LDPlayer 14 تفعيل تقنية الافتراضية على مستوى المعالج (VT-x لIntel أو AMD-V لAMD) في BIOS [32] [37]. كما أن متطلبات النظام معتبرة، حيث يوصى بشدة باستخدام معالج أو أفضل، مع 16 جيجابايت من ذاكرة الوصول العشوائي كحد أدنى، و32 جيجابايت موصى به للتشغيل متعدد الإثارات. هذه المتطلبات [37] [47]. AMD Ryzen 5 5600X أو Intel Core i5-12400

مصمم للأداء العالي، وهو ما يتناسب مع غرضه كأداة للألعاب LDPlayer 14 العالية تشير إلى أن التنافسية مثل PUBG Mobile. ومع ذلك، فإن هذا التركيز على الأداء قد يكون له جانبان؛ فمن ناحية، يوفر بيئة أكثر استقرارًا وسرعة لتشغيل اللعبة، ومن ناحية أخرى، قد يعني أن مضادات الاحتيال تكون أكثر كفاءة وقدرة على التفاعل مع النظام.

كأساس لافتراضية الجهاز الافتراضي VirtualBox يستخدم محرك LDPlayer الأهم من ذلك، أن هذا هو السبب الجوهري وراء التحذيرات الواردة في المصادر التي تفيد بأن . [48] الخاص به فالمحركات . [48] LDPlayer لن تعمل على GameLoop مثل QEMU الأدوات المخصصة لمحاكيات الافتراضية المختلفة لديها طبقات مختلفة من التعامل مع الذاكرة والجهاز، مما يؤدي إلى اختلافات في كيفية تنظيم ذاكرة التطبيق. وبالتالي، فإن أي إزاحة أو نمط تم العثور عليه في ذاكرة لعبة PUBG Mobile على GameLoop لن يكون له أي قيمة على LDPlayer. هذا يعني أن عملية البحث عن الإزاحات يجب أن تبدأ من الصفر لكل بيئة عمل جديدة، وهو ما يزيد من تعقيد وتكرار جهد البحث.

مجموعة أدوات قوية للتواصل LDPlayer للتغلب على هذه الفجوة وتسهيل عملية التطوير، يقدم وهي أداة قائمة بسطر، `ldconsole.exe` مع بيئته من خارج المحاكي. الأداة الأكثر أهمية هي تتيح هذه الأداة التحكم البرمجي الكامل في إثارات . [50] الأوامر موجودة في مجلد تثبيت المحاكي LDPlayer. يمكن استخدامها لتنفيذ مجموعة واسعة من الأوامر، بما في ذلك:

- `list2`: الحصول على قائمة بالمعلومات التفصيلية لجميع الإثارات النشطة، بما في ذلك VirtualBox الخاص بـ PIDالخاص بعملية المحاكي، و PID المؤشر الفريد، اسم العنوان، رقم PUBG Mobile بالتعرف التلقائي على عملية ESP هذه المعلومة حيوية لأنها تسمح لأداة . [51] وتحديد PID الخاص بها.
- `modify`: تعديل خصائص المحاكي بشكل ديناميكي، مثل الدقة الشاشة (`--resolution`)، وحتى التعديل على عدد نوى المعالج (`--cpu`)، كمية الذاكرة المخصصة (`--memory`) MAC Address و IMEI المعرفات الفريدة للجهاز مثل (`--imei`, `--mac`) [50] [51]. هذه القدرة على تجاوز الكشف عن المحاكي من خلال تزييف ESP القدرة على التكيف تمنح أداة هوية الجهاز.
- `adb`: تنفيذ أوامر ADB في أوامر ADB مباشرة على إثارة معينة، مما يدمج وظائف ldconsole [51].

فريدًا، والذي يتم حسابه ADB منفذ LDPlayer يعرض كل إثارة، `ldconsole.exe` بالإضافة إلى على سبيل المثال، الإثارة الأولى (الفهرس 0) . [42] [50] `5555 + index * 2` باستخدام الصيغة تستخدم المنفذ 5555، والإثارة الثانية (الفهرس 1) تستخدم المنفذ 5556. هذه القدرة على التفاعل عبر ADB تتيح للمطورين استخدام مجموعة واسعة من الأدوات والمكتبات المتوفرة لـ ADB، مثل `adb shell pm list packages` للعثور على حزم التطبيقات، أو `adb shell ps` للتحكم ADB كما يمكن استخدام . [12] اللعبة بدقة PID للحصول على قائمة العمليات وتحديد في الجهاز الافتراضي، مثل نقل الملفات (`adb push/pull`)، أو التقاط لقطات شاشة (`adb shell screencap`)، أو حتى محاكاة عمليات اللمس (`adb shell input tap`) [42].

يمكنها العمل بشكل تلقائي ESP يخلق أساسًا قويًا لأداة ADB و ldconsole.exe إن الجمع بين ومرن. بدلاً من الاعتماد على إعدادات ثابتة، يمكن للكود أن يكتشف البيئة المحيطة تلقائيًا. على ثم يبحث عن اسم اللعبة، ldconsole list2 سبيل المثال، يمكن كتابة برنامج يقوم بتشغيل "com.tencent.ig" في المخرجات، ويستخرج PID على بمجرد الحصول PID. العملية، يمكنه للحصول على خريطة الذاكرة، والتي تعد adb shell cat /proc/[pid]/maps استخدام هذا النهج .[12][46] وملفات أخرى مهمة libil2cpp.so بمثابة خريطة طريق لتحديد مواقع واستكشاف الذاكرة، مما يجعل العملية أكثر استقرارًا PID البرمجي يحل مشكلة العثور على وأتمتة.

على سبيل المثال، قد تكتشف .LDPlayer علاوة على ذلك، هناك تحديات أخرى مرتبطة ببيئة من خلال فحص خصائص النظام. تشمل هذه الخصائص LDPlayer اللعبة وجودها داخل Build.Hardware التي غالبًا ما تكون "goldfish" و Build.MODEL التي تكون "sdk" هذه .[22] يمكن للمطور، ldconsole modify علامات واضحة لوجود بيئة محاكاة. ومع ذلك، وبفضل بالإضافة .[51] تجاوز هذا الكشف عن طريق تغيير هذه الخصائص إلى قيم تشبه الأجهزة الحقيقية / والتي يمكن أن تمنع الوصول إلى SELinux، إلى ذلك، هناك إمكانية لمواجهة حواجز أمان مثل proc/[pid]/mem في مثل هذه الحالات، قد .[5] حتى لو كان النظام لديه صلاحيات الجذر في اللعبة نفسها لتجاوز هذه القيود Frida Gadget يتطلب الأمر تقنيات أكثر تقدمًا مثل حقن dinamically[39].

ليس مجرد خطوة أولى، بل هو عملية مستمرة LDPlayer 14 في المحصلة، فإن فهم بيئة والاستفادة من أدوات التحكم، (VirtualBox) ومتعددة الأوجه. يتضمن ذلك معرفة بنية المحرك والتكيف مع المتطلبات العالية للأداء، والتعامل مع الآثار ADB، و ldconsole.exe القوية مثل الجانبية للاستخدام (مثل الكشف عن المحاكي). إن استراتيجية النجاح لا تكمن في البحث عن حلول جاهزة من محاكيات أخرى، بل في بناء منصة مرنة تستطيع الاستشعار والتكيف مع بيئتها استكشاف وقراءة :ESP الخاصة، مما يوفر أساسًا متينًا للمرحلة التالية من عملية تطوير أداة ذاكرة اللعبة.

الخاصية	وصف	الأهمية للمطور
نظام التشغيل الأساسي	Android 14 [37] [38]	ضمان التوافق مع الألعاب الحديثة التي تتطلب SDK جديد.
محرك الافتراضية	VirtualBox [48]	يفسر عدم توافق الإزاحات والأنماط مع محاكيات مثل QEMU GameLoop.
دعم Hyper-V	دعم أصلي لوحدة تحكم Hyper-V (WHPX) [37] [47]	توفير إمكانية الوصول إلى ذاكرة المحاكي عبر النفق، مما يمثل طريقة قوية للتغلب على حواجز الذاكرة.
واجهة سطر الأوامر (`ldconsole.exe`)	أداة للتحكم البرمجي الكامل في الإثارات [50] [51]	القدرة على اكتشاف اللعبة، وتعديل خصائص المحاكي، وإدارة PID الإثارات بشكل تلقائي.
منفذ ADB	منفذ TCP 5555 + index * 2) فريد لكل إثارة [42] [50]	التفاعل مع الجهاز الافتراضي باستخدام أدوات ADB القياسية لاستكشاف الذاكرة والحصول على معلومات النظام.
متطلبات النظام	VT-x/AMD-V 16 +مفعل، معالج قوي، جيجابايت RAM [37] [47]	ضرورة أن يعمل النظام المستهدف بأداء عالٍ لتحقيق تجربة لعب سلسة وتجنب المشاكل.
كشف المحاكي	اكتشاف من خلال خصائص النظام (مثل `Build.Hardware`) [22]	تتطلب استراتيجية للتزييف باستخدام `ldconsole modify` لتعديل المعرفات الفريدة للجهاز.

آلية الوصول إلى ذاكرة اللعبة: من استكشاف الذاكرة إلى تفكيك بنية Il2Cpp

بعد تحديد بيئة LDPlayer 14 والتفاعل معها برمجيًا، ينتقل البحث إلى المرحلة الأكثر مركزية في تطوير أداة ESP: هذه العملية الوصول إلى بيانات اللعبة الموجودة في ذاكرة الوصول العشوائي. ليست مجرد قراءة عشوائية للذاكرة، بل هي رحلة منهجية تتطلب فهمًا دقيقًا لهيكل اللعبة، يتطلب هذا النهج عدة خطوات. Il2Cpp مع تقنية Unity وخاصةً عندما تكون مبنية على محرك متسلسلة: استكشاف هيكل الذاكرة الأولي، واستخراج البنية الهيكلية للعبة، ثم التغلب على حمايات الملفات التي تحتوي على هذه البنية.

كما تم .PUBG Mobile لعملية Process ID (PID) الخطوة الأولى والأكثر أهمية هي الحصول على توضيحه سابقًا، يمكن تحقيق ذلك بشكل موثوق وآلي باستخدام أداة `ldconsole.exe` مع الأمر `list2`، بمجرد الحصول على [51]. كل منها PID والتي تعرض قائمة بجميع الإثارات النشطة مع `/proc/[pid]/maps` تصبح البوابة مفتوحة لاستكشاف ذاكرة اللعبة. المفتاح هنا هو ملف، PID هذا الملف هو دليل الطرق [46]. Linux في (Procfs) الموجود في نظام الملفات خاص بالعملية الرقمي الذي يصف جميع مناطق الذاكرة المخصصة للعملية، بما في ذلك المكتبات المشتركة وملفات البيانات. يتكون كل سطر في هذا الملف من عدة أجزاء: نطاق (مثل `libil2cpp.so`) وبيانات الجهاز، والمسار إلى ،Offsetالعنوان الافتراضي، أذونات الوصول (قراءة، كتابة، تنفيذ)، ال فإن تحليل هذا الملف أمر حاسم لأنه يحدد المناطق، ESP بالنسبة لأداة [46]. Backing الملف الم القابلة للقراءة التي يمكن البحث فيها عن بيانات اللعبة، ويحدد أيضًا أسماء المكتبات الرئيسية

على سبيل المثال، سيحتوي الملف على سطر يشير إلى . [12] التي سيتم تحليلها لاحقًا libil2cpp.so، مما يوفر لنا المسار إلى هذا الملف داخل نظام الملفات الافتراضي للمحاكي.

نصل إلى جوهر المشكلة: فهم بنية libil2cpp.so، بمجرد تحديد موقع Il2Cpp. Il2Cpp هي تقنية تستخدمها Unity لتحويل الكود المكتوب بلغة C# إلى كود ثنائي (C++) لأغراض الأداء والأمان. على الرغم من هذا التحويل، فإن المعلومات الهيكلية الأصلية (أسماء الفئات، أسماء الحقول، نوع البيانات، وأحجامها) لا تزال موجودة في ملف أساسي آخر يسمى global-metadata.dat. [18] هذا الملف هو بمثابة "لوحة القيادة" التي تسمح للبرنامج بفهم بنية البيانات التي يتم تخزينها في ذاكرة الوصول العشوائية. بدون فهم هذا الهيكل، ستظهر بيانات الذاكرة مجرد أرقام عشوائية. لذلك، فإن استخراج وفك تشفير هذين الملفين هو الهدف النهائي لمرحلة استكشاف بنية اللعبة.

تتضمن عملية استخراج هذين الملفين عدة تحديات. أولاً، غالبًا ما يتم حماية ملفات اللعبة داخل APK/AAB. يتطلب ذلك فك ضغط حزمة التطبيق، وغالبًا ما يتم العثور على global-metadata.dat في مسار مثل assets/bin/Data/Managed/Metadata/global-metadata.dat. [18] ومع ذلك، فإن العديد من المطورين، بما في ذلك PUBG Mobile لـ Tencent، لا يضعون هذه الملفات ببساطة. قد يتم تخزينها في مكان مختلف، أو قد يتم إخفاؤها ثانيًا، قد تكون global-metadata.dat. [17] ضمن ملفات أخرى، أو قد يتم تغيير أسمائها مشفرة. على سبيل المثال، في ألعاب مثل Guardian Tales، يتم تخزين الملف في حالة مشفرة، مما يجعل libil2cpp.so مباشرة داخل global-metadata.dat ثالثًا، قد يتم إدراج . [17] وأخيرًا، قد يتم تغيير ترتيب حقول الملف بعد تحميله لجعل استخراجه منفصلًا أمرًا صعبًا . [18] القياسية تفشل arsersالب. [17]

للتغلب على هذه الحمايات المتعددة، يعتمد مجتمع تطوير المحسنات على مجموعة من التقنيات المتقدمة. الخطوة الأولى غالبًا ما تكون استخدام أداة مثل Frida لتعطيل آليات الحماية مؤقتًا أثناء تشغيل اللعبة. يمكن كتابة سكريبت Frida يقوم بإيقاف تشغيل ميكانيزمات التحقق من الت integrity، أو فك تشفير الملفات قبل أن يتم استخدامها بواسطة اللعبة. بمجرد تعطيل الحماية، يمكن استخدام أدوات مثل GameGuardian لنسخ الملفات مباشرة من الذاكرة أثناء تشغيل اللعبة. [8] هذه الطريقة تتجاوز معظم أنواع الحماية التي تعتمد على إخفاء الملفات أو تشفيرها.

بمجرد الحصول على ملفات libil2cpp.so و global-metadata.dat غير المحمية، يأتي دور Il2CppDumper، وهي أداة مفتوحة المصدر وقوية للغاية أصبحت معيار الصناعة لهذه المهمة. تقوم Il2CppDumper بتوليف هذه الملفات لاستخراج المعلومات الهيكلية وتخزينها في صورة [8] "DLL" تحتوي على تعريفات كاملة لهيكلة الفئات والهيكلات الموجودة (.h) وهمية وملفات هيدر على سبيل المثال، إذا كان لدى اللعبة فئة تسمى PlayerController في اللعبة. [16] [8] سينتج Il2CppDumper تعريفًا لـ C++ struct يشبه تحتوي على حقول m_Position و m_Health:

```
struct PlayerController {
    BaseClass base;
    int m_Health; // offset: 0x100
    Vector3 m_Position; // offset: 0x108
    // ... other fields
};
```

من قراءة الذاكرة بشكل ذكي. بدلاً من ESP هذه الملفات المولدة هي التي تمكن مطور أداة من `PlayerController` يقرأ عنوان كائن C++ قراءة بايتات عشوائية، يمكنه الآن كتابة كود للحصول على x108لقراءة الصحة أو 0 (100xمثل 0) ذاكرة اللعبة، ثم يضيف الإزاحة الصحيحة الموقع. إن وجود هذه التعريفات الدقيقة هو ما يحول القراءة العمياء للذاكرة إلى عملية موجهة وهادفة.

مع بيئات تحليل متقدمة مثل Il2CppDumper للمساعدة في عملية التحليل، يمكن دمج ملفات Ghidra. توفر Il2CppDumper سكريبت Python يسمى `il2cpp_header_to_ghidra.py` بالإضافة إلى . [16] فكه وتحليله Ghidra الناتج ويحوله إلى تنسيق يمكن لـ `il2cpp.h` يعالج ملف استيراد تعريفات الهياكل `ghidra_with_struct.py` ذلك، يمكن لسكريبت آخر يسمى مما يسمح بتصفح الكود المترجم من ،Ghidra والوظائف بشكل مباشر إلى قاعدة بيانات هذا يسهل . [16] مع عرض أسماء الحقول والفئات بدلاً من عناوين عشوائية `libil2cpp.so` بشكل كبير عملية تتبع تدفق البيانات وتحديد الأماكن التي يتم فيها تخزين وتعديل معلومات اللاعبين.

في بعض الحالات المعقدة، قد يلجأ المطورون إلى تقنيات أعمق. على سبيل المثال، قد يقوموا ثم استخدام ،(غير المقيدة Google باستخدام بناءات) على جهاز حقيقي `libart.so` بفك تشفير `libart.so` الحقلية، والتي يمكن بعد ذلك تطبيقها على offsets_لاستخراج DWARF معلومات هذا يوضح مدى التعقيد الذي يمكن أن يصل إليه الأمر، ولكنه . [5] المجمّع على الجهاز المستهدف يسلط الضوء على أن الطريق إلى بنية اللعبة ليس دائمًا سهلًا.

باختصار، آلية الوصول إلى ذاكرة اللعبة هي عملية متعددة المراحل تتطلب دقة ومهارة. إنها تبدأ ثم تتبع المسار إلى ،`ldconsole.exe` اللعبة من خلال أدوات مثل PID بالتعرف التلقائي على `/proc/[pid]/` من خلال تحليل `global-metadata.dat` و `libil2cpp.so` ملفات `maps`. لتجاوز الحمايات وفك تشفير Frida بعد ذلك، يتطلب الأمر استخدام تقنيات متقدمة مثل قابلة C++ لتحويل هذه الملفات إلى هيكلة Il2CppDumper الملفات، وأخيرًا، استخدام للاستخدام. كل خطوة في هذه السلسلة ضرورية، وإذا فشلت إحداها، فإن الوصول إلى بيانات اللعبة سيكون مستحيلاً.

المرحلة	الأداة/الطريقة	الهدف	التحديات الرئيسية
1. تحديد PID	`ldconsole.exe list2` أو `adb shell ps` [12] [51]	الحصول على معرّف عملية اللعبة لاستكشاف ذاكرتها.	ضرورة التفاعل مع المحاكي بشكل برمجي لتجنب الإعدادات الثابتة.
2. استكشاف الذاكرة	تحليل `/proc/[pid]/maps` [43] [46]	تحديد مناطق الذاكرة، وتحديد مواقع `libil2cpp.so` و `global-metadata.dat`.	فهم بنية الملف، وتحديد المناطق القابلة للقراءة.
3. استخراج الملفات	فك ضغط APK، GameGuardian، Frida [8] [18]	الحصول على نسخ غير محمية من `libil2cpp.so` و `global-metadata.dat`.	حماية الملفات (الإخفاء، التشفير، إعادة ترتيب الحقول) [17].
4. تفكيك البنية	Il2CppDumper [8] [16]	إنشاء تعريفات C++ (Heder file) لفئات اللعبة.	ضمان أن Il2CppDumper يعمل مع الإصدار المحدد من Unity وIl2Cpp.
5. التحليل المتقدم	Ghidra مع سكربت Il2CppDumper [16]	تحليل الكود المترجم وربطه بالتعريفات الهيكلية.	التعقيد الكبير للكود المولد من Il2Cpp.

تحديد نقاط الدخول: استراتيجيات البحث عن الإزاحات والأنماط

يواجه المطور تحديًا جديدًا ولكن لا يقل أهمية: تحديد الإزاحات الفعلية للحقول داخل هذه الهياكل. الإزاحة هي المسافة بالبايت من بداية كائن معين إلى البيانات التي يرغب في الوصول إليها، مثل صحة اللاعب أو موقعه. هذه الإزاحات ليست ثابتة؛ فهي تتغير مع كل تحديث للعبة، مما يجعل البحث عنها عملية مستمرة ومجهدة. لذلك، فإن استراتيجية النجاح لا تكمن في العثور على قائمة من الإزاحات الجاهزة، بل في فهم كيفية العثور عليها بنفسك بطريقة مرنة وقادرة على التكيف.

توجد عدة طرق رئيسية لتحديد هذه الإزاحات. الطريقة الأكثر شيوعًا وبدايةً هي استخدام أداة مثل Cheat Engine. تعمل هذه الأداة عن طريق البحث عن قيم معينة في الذاكرة مباشرة. على سبيل المثال، يمكن للمستخدم وضع صحة لاعب ما عند 100، ثم استخدام Cheat Engine للبحث عن القيمة 100 في ذاكرة اللعبة. بعد ذلك، عندما يفقد اللاعب صحته، يمكنه البحث عن القيمة الجديدة (مثلاً 95)، ومواصلة هذه العملية بشكل متكرر للتصفية حتى تبقى إزاحة واحدة فقط. هذه الطريقة فعالة جدًا للعثور على القيم البسيطة مثل الصحة أو النقود. ومع ذلك، فإنها تتطلب الوصول الخارجي إلى الذاكرة، وهو أمر قد يواجه صعوبات بسبب حواجز الذاكرة الخارجية التي تفرضها مضادات الاحتيال الحديثة [10]. بالإضافة إلى ذلك، فإنها لا تقدم فهمًا للبنية، مما يعني أن أي تحديث للعبة سيكسر هذه الإزاحات بشكل كامل.

للحصول على فهم أعمق وأكثر استدامة، يجب على المطورين الانتقال إلى تحليل ثابت للعبة باستخدام الملفات التي تم إنشاؤها بواسطة Il2CppDumper. في بمجرد تحميل تعريفات الهيكل

يمكن للمحلل تتبع الكود ليجد الوظائف التي تستخدم، Binary Ninja أو Ghidra بيئات تحليل مثل هذه الهياكل. على سبيل المثال، يمكن البحث عن استدعاءات إلى وظائف رسومية تأخذ كائنات كمدخلات، أو استدعاءات إلى وظائف تحسب المسافة بين لاعبين. بمجرد `PlayerController` العثور على وظيفة مهمة، يمكن تتبع كيفية استخدامها للحقول. على سبيل المثال، قد يرى المحلل تعليمات `mov eax, [ecx+0x48]`، مما يشير إلى أن الحقل الموجود عند الإزاحة 0x48 من كائن `PlayerController` يُقرأ. يمكن تكرار هذا العملية للحقول الأخرى مثل الموقع والاتجاه. هذه الطريقة تنتج إزاحات دقيقة وموثقة، ولكنها تتطلب خبرة كبيرة في التحليل الثنائي.

مع تطور مضادات الاحتيال، أصبحت الإزاحات الثابتة أقل موثوقية. لذلك، ظهرت استراتيجية أكثر تطورًا تركز على الأنماط. النمط هو تسلسل من البايتات (بما في ذلك أحرف و ILD) الذي يميز قطعة صغيرة من الكود الفريدة. على سبيل المثال، قد يكون هناك تسلسل مثل `E8 ? ? ? ? 8B 86` حيث تشير ? إلى أن البايت في تلك الموضع يمكن أن يكون أي شيء. هذه الوظيفة التي يحتوي عليها النمط غالبًا ما تكون قريبة من البيانات التي نريدها، ويمكن حساب الإزاحة النهائية من موقع النمط. هذه الاستراتيجية أكثر مرونة لأن النمط نفسه يميل إلى البقاء ثابتًا عبر التحديثات الصغيرة للعبة، بينما قد تتغير الإزاحات المطلقة. يمكن لأدوات مثل GH و ReClass.NET للمحلل بناء هيكل ذاكرة ديناميكي مساعدة في هذه العملية. يتيح ReClass.NET Offset Dumper قوية يمكن C++ فهو مكتبة، GH Offset Dumper أما [13]. وتصديره لاستخدامه في أدوات أخرى يمكنها [15]. دمجها في المشاريع الخاصة بالمطورين لمساعدتهم على إنشاء ملفات إزاحة آلية مسح الذاكرة للبحث عن الأنماط المحددة في ملف التكوين وحساب الإزاحات النسبية بشكل تلقائي.

لتطبيق هذه الاستراتيجيات، يمكن للمطورين اتباع نهج متدرج. يمكن البدء بالبحث عن الإزاحات في إصدار قديم من اللعبة أو في وضع تدريب. بمجرد العثور Cheat Engine البسيطة باستخدام بعد ذلك، يمكن .Ghidra في Thunk على هذه الإزاحات، يمكن استخدامها كنقطة انطلاق لتحليل إنشاء الأنماط للوظائف الرئيسية التي ترتبط بهذه الإزاحات. على سبيل المثال، يمكن إنشاء نمط يمكن كتابة دالة، ESP للوظيفة التي تقرأ صحة اللاعب، ووظيفة أخرى تقرأ موقعه. في كود أداة عند بدء التشغيل، ثم تحسب الإزاحة النهائية بناءً على `libil2cpp.so` تبحث عن هذا النمط في الفارق بين النمط والبيانات التي نريدها. هذه الطريقة تجعل الأداة أكثر مقاومة للتحديثات.

بالإضافة إلى اللاعبين، يجب العثور على الإزاحات للعناصر الأخرى في اللعبة. يشمل ذلك المركبات، والعناصر المتروكة، والأسلحة. تشير الأمثلة الكودية إلى أن هذه العناصر تقع في هيكل بيانات مشترك. على سبيل المثال، يمكن الوصول إلى صحة السيارة من خلال الإزاحة `vehicleCommon + 0x108` [23]. يمكن للمرء أن يفترض أن هناك قائمة أو مصفوفة من عناصر اللعبة في ذاكرة الوصول العشوائي، وكل عنصر يشير إلى كائن مختلف (لاعب، سيارة، صندوق ممتلكات). قد تحتوي هذه العناصر على حقل يحدد نوعها (على سبيل المثال، `itemType`). يمكن استخدام هذا النوع للتمييز بين اللاعبين والمركبات واللوازم. على سبيل المثال، في الكود المقدم، `itemType=8` يشير إلى اللاعب، و `itemType=5` يشير إلى المركبة. يمكن للمطور كتابة منطق يتحقق، `itemType` بمجرد العثور على الإزاحة التي تحتوي على [48]. من هذا النوع، ثم يقرأ الحقول المناسبة بناءً على النوع. على سبيل المثال، إذا كان النوع 8

إذا كان النوع 5 (مركبة)، فإنه يقرأ الصحة من الإزاحة `entityAddv + 0x77C`، (لاعب) يقرأ صحة المركبة من الإزاحة `vehicleCommon + 0x108`.

إن عملية تحديد نقاط الدخول هي في جوهرها عملية استكشاف. تتطلب الصبر والاهتمام بالتفاصيل والقدرة على استخدام مجموعة متنوعة من الأدوات. إن النهج الأكثر فعالية هو الجمع بين البحث الديناميكي (Cheat Engine) والتحليل الثابت (Ghidra) واستخدام الأنماط للحصول على حلول مرنة. يجب أن يُبنى كود أداة ESP والأنماط ESP على إدارة هذه الإزاحات نظام قادر على تحديث للعبة. بدون هذه الطريقة المنهجية، فإن أي أداة ESP مع آلية لتحديثها بسهولة كلما حدث ستشكل عرضة للانهيار بعد أول تحديث بسيط للعبة.

نوع البيانات	الإزاحة النسبية (مثال)	الوظيفة المقترحة	الأداة المستخدمة للعثور عليها
موقع اللاعب	`NavMesh + 0x150` [23]	قراءة إحداثيات X, Y, Z للعالم.	Cheat Engine, Ghidra
صحة اللاعب	`entityAddv + 0x77C` [23]	قراءة قيمة الصحة.	Cheat Engine
طاقة اللاعب	`entityAddv + 0x1408` [23]	قراءة قيمة الطاقة.	Ghidra (تحليل الكود)
معرف الفريق	`entityAddv + 0x610` [23]	قراءة معرف الفريق (友軍/敵軍).	Ghidra (تحليل الكود)
حالة اللاعب	`playerWorld + 0x130` [23]	قراءة حالة اللاعب (standing, crouching, etc.).	Ghidra (تحليل الكود)
صحة المركبة	`vehicleCommon + 0x108` [23]	قراءة صحة المركبة.	Ghidra (تحليل الكود)
وقود المركبة	`vehicleCommon + 0x120` [23]	قراءة كمية الوقود.	Ghidra (تحليل الكود)
معرف الكلاس	`itemType` [48]	تحديد نوع العنصر (لاعب، مركبة، لواء).	Ghidra (تحليل الكود)

التنفيذ البرمجي الأساسي: حقن الكود وقراءة البيانات

بعد استكشاف بنية اللعبة وتحديد الإزاحات والأنماط، ينتقل المشروع إلى مرحلة التنفيذ البرمجي الأساسية: بناء الجزء الذي يقرأ البيانات من ذاكرة اللعبة. هذه المرحلة هي الأكثر تقنية، حيث يتم دمج كل ما تم تعلمه في كود قابل للتنفيذ. يتكون هذا الكود عادةً من جزأين: آلية لحقن الشيفرة في عملية اللعبة، والشيفرة نفسها التي تقوم بقراءة الذاكرة وتخزين البيانات اللازمة.

هذه التقنية تسمح ".DLL تتمثل الطريقة الأكثر شيوعًا وقوةً لحقن الكود في استخدام تقنية "حقن مخصص داخل مساحة عنوان عملية PUBG Mobile [6]. لأداة ESP الخارجية بتحميل ملف DLL تعمل العملية كالتالي: يستخدم البرنامج الخارجي وظائف Windows API مثل `CreateRemoteThread` لإنشاء خيط جديد داخل عملية اللعبة. هذا الخيط الجديد يُعطَى مهمة `LoadLibrary`، لتنفيذ وظيفة لكنه يمرر إلى هذه الوظيفة مسار ملف DLL الخاص بأداة ESP. في ذاكرة DLL داخل عملية اللعبة، يقوم نظام التشغيل بتحميل `LoadLibrary` بمجرد تنفيذ تثبيت DLL الخاصة به عند التحميل. من هنا، يمكن لـ `DllMain` بتنفيذ دالة DLL اللعبة، ويقوم

المزيد من التعديلات، مثل تثبيت مثبتات على وظائف معينة، أو بدء خيوط قراءة البيانات الخاصة به.

هناك العديد من التقنيات البديلة لحقن DLL، ولكل منها مزاياه وعيوبه. "Process Hollowing" هي تقنية أكثر تعقيدًا حيث يتم إنشاء عملية جديدة في حالة معلقة، ثم يتم تفريغ ذاكرة (RunPE) العملية من المExecutable الأصلي وكتابة م executable الخاص بأداة ESP في مكانه، ثم استئناف العملية. هذه الطريقة تجعل العملية تبدو وكأنها لعبة حقيقية [6]. تقنية أخرى هي "APC Injection"، حيث يتم إضافة دالة (مثل `LoadLibrary`) إلى قائمة APCs للThreads المستهدفة، مما يؤدي إلى تنفيذها عندما يدخل الخيط في حالة alertable [6]. اختيار التقنية يعتمد على مدى فعالية معروف بقدرته على BattlEye، مضاد الاحتيال في الكشف عن هذه الطرق. على سبيل المثال، كشف حقن DLL القياسي [21].

بمجرد نجاح الحقن، يبدأ الكود داخل عملية اللعبة مهمته الأساسية: قراءة الذاكرة. هذا يتطلب معرفة بـ PID للعبة (الذي يمكن الحصول عليه من خلال وظائف Win32 API مثل `GetCurrentProcessId`)، والمسار إلى `libil2cpp.so` (الذي يمكن العثور عليه في ذاكرة اللعبة نفسها)، والإزاحات التي تم تحديدها سابقًا. تبدأ العملية بقراءة عنوان الـ基座 (base address) لـ `libil2cpp.so` من ملفات الروابط `/proc/self/maps` استخلاصه من داخل عملية اللعبة نفسها). بعد ذلك، يتم حساب عنوان كائن اللاعب الأول أو اللاعب الحالي عن طريق جمع العنوان الأساسي مع الإزاحة المعروفة للقائمة. بعد ذلك، يتم قراءة عنوان اللاعب التالي من اللاعب الحالي، وهكذا، لتمرير القائمة. بمجرد الحصول على عنوان كائن اللاعب، يمكن قراءة بيانات مثل الموقع والصحة باستخدام الإزاحات المقابلة. على سبيل المثال، لقراءة موقع اللاعب، والتي ستكون `playerAddress + offset_PlayerPosition`، سيقرأ الكود القيمة عند هذا العنوان. يجب تكرار هذه العملية للحقول المدمجة، مثل قراءة `Vector3` هي نفسها عنوان كائن آخر. يجب أن تكون هذه العمليات `playerAddress + offset_PlayerHealth`. الصحة من سريعة قدر الإمكان لتجنب إبطاء اللعبة. يمكن استخدام وظائف مثل `ReadProcessMemory`، في مثل هذه [10]. ولكن يجب ملاحظة أنه قد يتم حظر هذه الوظائف من قبل مضادات الاحتيال الحالات، قد يتطلب الأمر استخدام تقنيات أكثر تقدمًا مثل حقن Frida Gadget، والذي يوفر وظائف قراءة الذاكرة الخاصة به من داخل عملية اللعبة [43].

لتحسين الأداء وتجنب انخفاض معدل الإطارات، من الأفضل تفكيك عملية قراءة الذاكرة عن عملية الرسم. يمكن تحقيق ذلك باستخدام خيوط متعددة. يمكن إنشاء خيط قراءة مخصص يقوم بدوران مستمر (في حلقة `while(1)`) لقراءة بيانات اللاعبين والمركبات من الذاكرة وتخزينها في "مخبأ" مشترك (shared buffer) يمكن للخيط الرسومي الآخر الوصول إليه، أو هيكل بيانات في الذاكرة. هذا يضمن أن قراءة الذاكرة البطيئة لا تؤثر على سلاسة الرسوميات ورسمها على الشاشة [3]. يجب استخدام آليات تزامن (مثل Mutexes أو Critical Sections) لحماية المخبأ المشترك من الوصول المتزامن من خطي القراءة والرسم.

أخيرًا، يجب أن يتعامل الكود مع بعض المشكلات الهندسية الأساسية. أحد أهمها هو تحويل إحداثيات العالم ثلاثية الأبعاد (X, Y, Z) إلى إحداثيات شاشة ثنائية الأبعاد (X, Y). هذا يتطلب معرفة بمصفوفات العرض والرؤية والإنشاء التي تحدد كيفية تحويل 3D world coordinates to 2D screen coordinates [1]. تتضمن العملية عادةً ضرب إحداثيات اللاعب الثلاثية الأبعاد بمصفوفة Perspective (العرض، ثم تطبيق القسمة الو قسمة الإحداثي W)، ثم م Normalization الإحداثيات ثم تحويلها إلى نظام الإحداثيات الطبيعي المعدل (Normalized Device Coordinates) إحداثيات بكسل بناءً على دقة الشاشة. هذه العمليات الرياضية معقدة وقد تتطلب دقة عالية لإجراء هذه العمليات GLM (OpenGL Mathematics) لتجنب الأخطاء. يمكن استخدام مكتبات مثل بشكل فعال [1].

بالإضافة إلى ذلك، يجب على الكود التعامل مع "الإطارات العظمية" للشخصيات. هذه هي الإطارات التي تحتوي على مواقع عظام الرأس والصدر وغيرها. يمكن العثور على هذه الإطارات باستخدام الإزاحات المماثلة للعناصر الأخرى. بعد قراءة مواقع العظام، يمكن رسم خطوط رؤية للتأكد من (Standing, Crouched, Prone) أو صناديق حولها. يجب أيضًا التعامل مع حالة اللاعب أن الأبعاد المحسوبة صحيحة.

إنه المسؤول عن الاتصال .ESP في نهاية المطاف، يمثل هذا الجزء البرمجي "القلب" لأداة المباشر مع ذاكرة اللعبة، واستخراج البيانات الخام، وتحويلها إلى معلومات مفيدة. تصميم هذا القسم بعناية، مع التركيز على الأداء، والمرونة، وقابلية الصيانة، هو ما يفرق بين أداة بسيطة وسريعة الكسر، وأداة قوية ومستدامة.

المكون	الوصف	التقنيات المستخدمة	التحديات الرئيسية
آلية الحقن	دمج الشيفرة في عملية اللعبة.	حقن DLL عبر `CreateRemoteThread`, Process Hollowing, APC Injection [6].	كشف مضاد الاحتيال (Handle Protection, Manual Map Detection) [10] [21].
قراءة الذاكرة	استرجاع البيانات من ذاكرة اللعبة.	`ReadProcessMemory` (قد تكون محظورة)، حقن Frida Gadget، أو الوصول المباشر من داخل العملية.	الأداء (تجنب انخفاض FPS)، التعامل مع حواجز الذاكرة.
إدارة البيانات	تخزين البيانات المستخرجة.	مخبأً مشتركًا (Shared Buffer) بين خيط القراءة وخيط الرسم.	تزامن الخيوط (Mutexes, Critical Sections) لتجنب مشاكل الحالة الطارئة.
معالجة الإحداثيات	تحويل إحداثيات العالم 3D إلى إحداثيات الشاشة 2D.	ضرب المصفوفات (المصفوفة العرضية والمصفوفة الإنشائية) والقسمية الو Perspective [1].	التعقيد الرياضي، دقة الحسابات، التعامل مع زوايا الرؤية المختلفة.
البنية العامة	بنية الكود.	استخدام خيوط متعددة (Reader Thread, Renderer Thread) [3].	إدارة حالة الخيوط، الذاكرة المشتركة، تكامل المكونات.

الرسومات والعرض: بناء واجهة المستخدم المرئية وإدارة الأداء

بعد أن يقوم الجزء الأساسي من الكود بجمع البيانات من ذاكرة اللعبة، تأتي المرحلة التالية وهي الأكثر تفاعلًا مع المستخدم: عرض هذه البيانات بشكل مرئي على الشاشة. يتم ذلك عادةً من خلال إنشاء "نافذة معلقة" فوق نافذة اللعبة. هذه النافذة هي التي ترسم عناصر مثل خطوط الرؤية، والصناديق حول اللاعبين، وأسمائهم، ومستويات صحتهم، ومسافاتهم. إن تصميم هذه الطبقة المرئية ليس مجرد مسألة جمالية؛ فهو يتعلق أيضًا بالأداء وسرية الأداة.

الخيار المفضل لإنشاء هذه النوافذ المعلقة. تسمح DirectX 11 (D3D11) تُعد تقنية D3D11 بإنشاء نافذة خاصة يمكن تكوينها لتكون شبه شفافة وغير متداخلة مع النقرات (click-through). هناك العديد من الإطارات العمل الجاهزة التي تسهل هذه العملية. على سبيل المثال، مشروع "D3D11 Overlay ImGui" على GitHub هو إطار عمل ممتاز يجمع بين D3D11 لإنشاء السور الواقي و ImGui. ImGui هو مكتبة رسم UI بسيطة لإنشاء واجهة المستخدم التفاعلية (ImGui) [2] ImVecN، وسريعة تسمح لك بكتابة كود C++ لإنشاء عناصر واجهة المستخدم مثل النوافذ، والمربعات، والنوافذ المنبثقة، والتي يتم ترجمتها إلى رسومات في الوقت الفعلي. هذا يسمح للمطور بإنشاء أدوات تحكم بسيطة لـ ESP (مثل تشغيل/إيقاف تشغيل الميزات، أو ضبط الحساسية) دون الحاجة إلى بناء واجهة مستخدم معقدة من الصفر.

تتطلب عملية إنشاء نافذة معلقة باستخدام D3D11 عدة خطوات. أولاً، يتم إنشاء نافذة باستخدام DirectX 11 بدون أي إطار أو عنوان. ثانيًا، يتم إعداد سياق WS_POPUP الخاص بالنافذة. ثالثًا، يتم إنشاء سمة نافذة شبه شفافة. الطريقة الأكثر فعالية لهذه الغاية هي استخدام وظيفة Windows API DwmExtendFrameIntoClientArea مع هامش -1، والتي تجعل النافذة تتوسع لتشمل منطقة العميل بالكامل وتسمح لها بالتدفق بشكل طبيعي [3]. هذا يوفر شفافية أفضل من استخدام خاصية WS_EX_LAYERED التي قد تسبب حواف بيضاء. بعد ذلك، يمكن للكود الرسومي الذي يعمل في خيط منفصل (لتحسين الأداء) قراءة البيانات من المخبأ المشترك (الذي يملأه خيط القراءة) ثم استخدام ImGui لرسم عناصر ESP. على سبيل المثال، يمكن للبرنامج قراءة قائمة باللاعبين الحاليين، ثم لجميع اللاعبين في القائمة، إذا كانوا في الأفق، قراءة موقعهم على الشاشة، ثم رسم اسمهم وصندوق حولهم في تلك الإحداثيات.

تحسين الأداء هو أمر بالغ الأهمية. كما ذكرنا سابقًا، فإن فصل قراءة الذاكرة عن الرسم إلى خيطين منفصلين هو أفضل ممارسة [3]. على سبيل المثال، يجب أن يعمل خيط القراءة بتردد منخفض (على سبيل المثال، 50 هرتز)، بينما يجب أن يعمل خيط الرسم بتردد عالٍ (على سبيل المثال، 1000 هرتز) لضمان استجابة سلسة. يجب أيضًا تقليل كمية الرسومات التي يتم رسمها. على سبيل المثال، لا يجب رسم أي شيء إلا إذا كان اللاعب في الأفق. يجب أيضًا تقليل عدد الطلبات الرسومية. بدلاً من رسم كل عنصر على حدة، يمكن 모아 many requests into a single batch whenever possible. بالإضافة إلى ذلك، يجب استخدام QualitySettings.maxQueuedFrames في Unity (إذا كان ذلك ممكنًا) لتحديد عدد الإطارات القابلة للانتظار بين GPU و CPU، مما يمنع تراكم الأوامر ويقلل من تأخير الإدخال [30].

أما بالنسبة للسرية، فقد تطورت تقنيات التخفي بشكل كبير. في الماضي، كان الهدف هو إخفاء نافذة ESP عن Anti-Cheats. كانت تقنية شائعة هي "استغلال النافذة (Window Hijacking)، وتعديل بنية (مثل Discord overlay أو RTSS) بإيجاد نافذة موثوقة بالفعل cheat حيث يقوم cheatالخاصة بال render الخاصة بها في الذاكرة لجعلها تشير إلى وظائف `tagWND` هذا .[4] كنافذة من تطبيق آخر. ومع ذلك، فإن هذه التقنية معقدة للغاية وتتطلب استغلالاً cheat يخفي اليوم، يعتبر الكثير من .[4] النوافذ enumerating للنواة لتجنب الكشف عن طريق الكشف عن بدلاً من .[3] الخبراء أن هذه التعقيدات لا تبرر الفوائد الأمنية، خاصة مع ظهور أساليب أكثر تطورًا ذلك، يركز المطورون على أساليب أكثر بساطة وفعالية. على سبيل المثال، يمكن استخدام كما يمكن .[3] كما ذكرنا للحصول على شفافية نظيفة `DwmExtendFrameIntoClientArea` ما لم يكن المؤشر فوقها، مما يجعلها غير (alpha = 0) تكوين النافذة لتكون شبه شفافة تمامًا عن OBS مرئية للمنافسين. كما يمكن إخفاء واجهة المستخدم من برامج التقاط الشاشة مثل طريق تكوين D3D11 لاستخدام خاصية معينة لذلك .[2]

أن تكون قادرة على التعامل مع التحديات المتعلقة ESP بالإضافة إلى ذلك، يجب على أداة بالشاشة. بعض مضادات الاحتيال تقوم بقطع شاشة دوريًا للبحث عن نوافذ معلقة. يمكن أن منفصلة في تقليل فرص DX11 بدلاً من إنشاء نافذة DWM/Composition يساعد استخدام يمكن أن "Beamracing" كما أن التحكم الدقيق في التأخير أمر حاسم. تقنيات مثل .[4] الكشف تقلل من تأخير الإدخال إلى حد كبير عن طريق محاكاة مسار الرنين الرأسي للشاشة، مما يسمح Beamracing على الرغم من أن تطبيق .[27][29] للعبة بالرد على الإدخالات في أقرب وقت ممكن قد يكون صعبًا، إلا أن فهم مبادئه يسلط الضوء على الأهمية القصوى LDPlayer بشكل كامل مع للحد من التأخير.

في الختام، تشكل طبقة الرسومات والعرض جسرًا بين البيانات الخام من ذاكرة اللعبة والمعلومات التي يمكن للمستخدم استخدامها. يتطلب بناؤها توازنًا دقيقًا بين الأداء، والوظائف، يمكن للمطورين بناء واجهات مستخدم مرنة ،ImGui و D3D11 والسرية. باستخدام تقنيات مثل وسريعة. ومع ذلك، فإن الاستمرارية تأتي من تطبيق ممارسات الأداء الجيدة، مثل استخدام خيوط متعددة، والرسومات الذكية، والتقنيات البسيطة للتخفٍّ التي لا تعتمد على استغلال ثغرات معقدة في نظام التشغيل.

المكون	الوصف	التقنيات المستخدمة	التحديات الرئيسية
إنشاء النافذة	إنشاء نافذة شفافة وغير متداخلة فوق اللعبة.	D3D11, `DwmExtendFrameIntoClientArea` [3], `WS_POPUP` style.	التوافق مع مضادات الاحتيال التي تبحث عن نوافذ topmost أو layered.
مكتبة الرسومات	مكتبة لإنشاء واجهة المستخدم ورسم العناصر.	ImGui (ImVecN) [2].	التعلم، والتكامل مع D3D11.
الأداء	ضمان عدم انخفاض معدل الإطارات (FPS) أو زيادة التأخير.	تقليل الطلبات الرسومية، [3] خيوط متعددة (قرصان/كاتب)، تقليل الرسومات غير الضرورية.	التوازن بين تكرار تحديث البيانات وتأثيره على الأداء.
السرية	إخفاء نافذة ESP عن مضادات الاحتيال والمنافسين.	`DwmExtendFrameIntoClientArea` [3]، شفافية كاملة (alpha=0)، إخفاء من برامج التقاط الشاشة [2].	تطور مضادات الاحتيال في كشف النوافذ المعلقة (التصوير الشاشي، التحقق من تعداد النوافذ).
معالجة الإحداثيات	تحويل إحداثيات العالم 3D إلى إحداثيات الشاشة 2D.	ضرب المصفوفات (العرض، الإنشاء)، القسمية الو Perspective [1].	التعقيد الرياضي، دقة الحسابات، التعامل مع زوايا الرؤية المختلفة.

وتقنيات ACE الدفاعات والتغلب عليها: فهم نظام التخفي المتقدمة

إن تطوير أداة ESP ناجحة لا يقتصر على قدرة القراءة والعرض، بل يكمن بشكل أساسي في القدرة على التغلب على الدفاعات المعقدة التي تفرضها اللعبة وبيئتها. PUBG Mobile، مثل معظم الألعاب التنافسية الكبرى، يستخدم مجموعة من المكونات المتقدمة لمكافحة الاحتيال لحماية بيئة اللعب. إن فهم هذه الدفاعات وتطوير استراتيجيات للتغلب عليها هو ما يفصل بين أداة قابلة للاستخدام مؤقتًا وأداة قوية ومستدامة.

ACE ليس مجرد أداة واحدة، بل هو نظام متكامل يوفر حماية متعددة الطبقات. فهو ي Encrypt الملفات `.so` و `.dll`. فإنه يعزز Unity، بالنسبة للألعاب القائمة على [20]. الرئيسية للعبة لمنع التحليل الثابت DLLs عن طريق تشفير أسماء المتغيرات والدوال والصفوف، مما يجعل من الصعب للغاية حماية [20]. علاوة على ذلك، يوفر Ghidra تتبع تدفق البيانات باستخدام أدوات مثل [20]. هذا يعني أن أي محاولة التجسس والقرصنة، ويمنع أي محاولات لتعديل الموارد داخل اللعبة بشكل مباشر ستواجه تحديات `global-metadata.dat` أو `libil2cpp.so` لفك تشفير كبيرة.

تم تصميم هذا الدفاع الثاني في أواخر عام 2023 "Anti-ESP" الذي تم تقديمه رسميًا [24]. يعمل على تقليل Anti-ESP وبدلاً من مجرد حظر اللاعب، فإن النظام تحديدًا لمواجهة أداة ESP. ESP عن طريق إزالة تفاصيل الموقع للاعبين الذين لا يرونهم. هذا يعني أن استخدام ESP فائدة قديم أو غير محدث سيؤدي فقط إلى ظهور رمزات غير دقيقة أو لا تتحرك على الإطلاق، مما حيث يجب أن تكون محدثة باستمرار، ESP يجعله عديم الفائدة تقريبًا. هذا يمثل تحديًا مباشرًا لأداة

تعمل على تحسين هذا النظام باستمرار، مما Tencent كما أن Anti-ESP. لتتوافق مع كيفية عمل يعني أن المطورين يجب أن يكونوا على دراية بالتحديثات القادمة [24].

بتحليل بيانات Tencent الدفاع الثالث، والذي يمثل التهديد الأكبر، هو فحص سلوك الخادم. تقوم التي يتم إرسالها إلى خوادمها. تشير التقارير إلى أنهم يراقبون (in-game data) 中游戏ت وتوقيت ردود الفعل (flick speed)، إحصائيات مثل نسبة إصابات الرأس، وسرعة التحوّل (reaction time) [44]. أي نشاط يبدو غير طبيعي بشدة (مثل القدرة على إطلاق النار بدقة متناهية في أجزاء صغيرة من الثانية) سيتم تصنيفه على أنه مزيف. الأنشطة التي تتطلب تلاعبًا بالفيزياء، الهدف بدون الحاجة (مثل "Silent Aim" أو (رصاصة تتغير مسارها بعد إطلاقها) "Magic Bullet" إلى محاذاة)، تُعتبر خطيرة للغاية لأنها تخلق "آثارًا" واضحة في سجلات الخادم التي يمكن اكتشافها بسهولة [44]. لذلك، فإن استراتيجية النجاح لـ "يجب أن تركز على ميزات "الكشف ESP التي لا تتطلب تلاعبًا بالفيزياء، مثل عرض الأسماء والمسافات والخطوط المتقاطعة (ESP).

بسياسات "التسامح PUBG Mobile بالإضافة إلى هذه الدفاعات، هناك دفاعات أخرى. تشتهر هو المحاكي GameLoop على الرغم من أن الصفري" تجاه استخدام المحاكيات غير الرسمية [19]. إجراءات صارمة ضد اللاعبين Tencent قد لا يزال يواجه انتقادات. تتخذ LDPlayer الرسمي، إلا أن الذين يستخدمون محطات عمل غير معتمدة، والتي قد تؤدي إلى حظر لمدة تتراوح بين سنة إلى 10 سنوات [19]. هذا يفرض على أداة ESP أن تكون قادرة على التستر على نفسها داخل يمكن أن يشمل ذلك تعديل خصائص الجهاز (مثل LDPlayer. و `Build.MODEL` لجعل اللعبة تعتقد أنها تعمل على جهاز باستخدام `ldconsole modify` (`Build.Hardware` محمول حقيقي [22] [51].

تبني استراتيجية متعددة الطبقات للتغلب على ESP لتلبية هذه التحديات، يجب على مطور أداة الحواجز. أولاً، عند الوصول إلى الذاكرة، إذا كانت مضادات الاحتيال تمنع الوصول الخارجي (مثل `PROCESS_VM_READ`) [10]، باستخدام Vanguard لحذف أذونات `ObRegisterCallbacks` أو حقن DLL فإن الحل الوحيد هو الحقن الداخلي. كما ذكرنا سابقًا، تقنيات مثل حقن Frida على وجه الخصوص، قوي للغاية لأنه Frida Gadget، داخل عملية اللعبة هي ضرورية Gadget. على سبيل المثال، يمكن لسكريبت Frida يمكنه تعطيل العديد من آليات الحماية dinamically. نفسه، أو تعطيل الكشف عن وجود جذر، أو حتى تعطيل وظائف Frida تعطيل الكشف عن وجود مباشرة Anti-Cheat [39] [43].

لتعطيل الكشف عن التلاعب. على سبيل Frida يمكن استخدام ACE، ثانيًا، للتغلب على حماية المثال، يمكن إيجاد وظائف تتحقق من سلامة الملفات وتثبيت مثبتات عليها لإجبارها على العودة مباشرة `global-metadata.dat` لفك تشفير Frida بقيمة ناجحة دائمًا. يمكن أيضًا استخدام بدون الحاجة إلى العثور على Il2CppDumper من الذاكرة أثناء تشغيل اللعبة، مما يوفر بيانات الملف المشفر في ملف APK [8]. كما أن استخدام تقنيات Hypervisor مثل أو hyper-reV يمكن أن توفر طريقة قوية للتغلب على حواجز الذاكرة، WHPX التي تستخدم واجهة WinVisor، نظام التشغيل الضيف بالكامل bypass الخارجية، حيث أنها تعمل على مستوى النفق وت [54] [55].

نظيفة". هذا يعني تجنب أي نشاط قد يثير الشكوك. على" ESP ثالثًا، للتخفٍّ، يجب أن تكون أداة سبيل المثال، يجب تجنب أي كود يتحقق من وجود مضادات الاحتيال أو يحاول إيقاف تشغيلها. يجب أن يكون حقن الكود خفيًا تمامًا. كما ذكرنا، فإن تقنيات التخفي المتقدمة مثل استغلال النافذة معقدة وغير ضرورية. بدلاً من ذلك، يجب الاعتماد على أساليب بسيطة وفعالة مثل إنشاء نافذة شفافة باستخدام DWM و D3D11 [3]، وإخفاء واجهة المستخدم عند عدم الحاجة إليها، وعدم القيام بأي أنشطة غير ضرورية في الخلفية.

في الختام، يعد التغلب على الدفاعات مجالًا حيويًا ومتطورًا باستمرار. لا يوجد حل سحري واحد. يجب على المطور أن يكون على دراية بجميع طبقات الحماية التي يستخدمها PUBG Mobile (ACE، Anti-ESP، فحص سلوك الخادم) وأن يطور مجموعة من التقنيات للتغلب عليها. يعتمد النجاح على الجمع بين الحقن الداخلي، والبرمجة الديناميكية باستخدام تقنيات Frida، واستخدام ESP التخفي البسيطة والفعالة، مع التركيز على ميزات التي لا تنتهك قواعد السلوك السلوكي للعبة. إن فهم هذه البيئة التنافسية هو ما يضمن بقاء أداة ESP قابلة للاستخدام على المدى الطويل.

الدفاع	الوصف	الاستراتيجية للتغلب عليه	الأدوات/التقنيات
ACE (Anti-Cheat Engine)	حماية متعددة الطبقات تشمل تشفير الملفات، وحماية الموارد، ومنع التجسس [20].	حقن الشيفرة Dinamically لتعطيل الحماية، أو فك تشفير الملفات من الذاكرة أثناء التشغيل [8].	Frida (لتعطيل الحماية)، حقن DLL/ Process Hollowing [6].
Anti-ESP	نظام يلغي تفاصيل موقع اللاعبين الخارجين من الرؤية لجعل ESP عديم الفائدة [24].	تحديث الأداة باستمرار لمواكبة كيفية عمل Anti-ESP. الاعتماد على ميزات "الكشف" (ESP) فقط.	بحث مستمر، تحديث الإزاحات والأنماط.
فحص سلوك الخادم	مراقبة البيانات المرسلة إلى الخادم (مثل نسبة إصابات الرأس، سرعة التحوّل) [44].	تجنب الميزات التي تتلاعب بالفيزياء (مثل Magic Bullet, Silent Aim). التركيز على ميزات ESP البحتة.	برمجة ذكية، تجنب التلاعب بالفيزياء.
اكتشاف المحاكي	اكتشاف أن اللعبة تعمل داخل LDPlayer من خلال فحص خصائص النظام [22].	تعديل خصائص الجهاز (مثل `Build.MODEL`) لجعلها تبدو كجهاز حقيقي [51].	`ldconsole modify` (تغيير IMEI, MAC, إلخ).
حجب الذاكرة الخارجية	منع الوصول الخارجي إلى ذاكرة العملية باستخدام واجهات Kernel Callbacks (مثل `ObRegisterCallbacks`) [10].	استخدام الحقن الداخلي (In-process injection) لقراءة الذاكرة من داخل عملية اللعبة.	حقن DLL، حقن Frida Gadget، حقن APC [6].
الكشف عن النافذة المعلقة	كشف مضادات الاحتيال لنوافذ ESP غير الخفية أو غير القياسية.	استخدام نوافذ شفافة ومخفيّة (مثل `DwmExtendFrameIntoClientArea`) [3]. تجنب تقنيات الاستيلاء المعقدة.	D3D11 Overlay with ImGui [2].

مرجع

1. Arcore 3d coordinate system to 2d screen coordinates https://stackoverflow.com/questions/73372680/arcore-3d-coordinate-system-to-2d-screen-coordinates
2. D3D11 Overlay with ImGui (x86 / x64) - EXE / DLL DirectX ... https://github.com/adamhlt/D3D11-Overlay-ImGui
3. [Help] Good Method to draw imgui external ? https://www.unknowncheats.me/forum/counter-strike-2-a/624372-method-draw-imgui-external.html
4. [Question] Reliable Stealth Overlay Methods for PUBG (2026) https://www.unknowncheats.me/forum/anti-cheat-research/748954-reliable-stealth-overlay-methods-pubg-2026-a.html
5. Assessing the Forensic Viability of Android Memory ... https://arxiv.org/html/2607.13821v1
6. Ten process injection techniques: A technical survey of ... https://www.elastic.co/blog/ten-process-injection-techniques-technical-survey-common-and-trending-process
7. Support for Memory Reading and Writing in Android ... https://github.com/ufrisk/MemProcFS/issues/362
8. Perfare/Il2CppDumper: Unity il2cpp reverse engineer https://github.com/perfare/il2cppdumper
9. How to use Il2CppDumper to reverse Unity IL2CPP games (2020) https://www.youtube.com/watch?v=gc3I_u_Xy4c
10. How Kernel Anti-Cheats Work: A Deep Dive into Modern ... https://s4dbrd.github.io/posts/how-kernel-anti-cheats-work/
11. Emulators in Gaming: Threats and Detections | AppSec Articles https://docs.talsec.app/appsec-articles/articles/emulators-in-gaming-threats-and-detections
12. [Coding] read memory of .lib from emulator https://www.unknowncheats.me/forum/general-programming-and-reversing/625118-read-memory-lib-emulator.html
13. Finding Game Offsets with ReClass.NET https://paste-co.de/guides/finding-offsets-reclass
14. How To Find Offsets, Entity Addresses & Pointers https://www.youtube.com/watch?v=YaFlh2pIKAg
15. guidedhacking/GH-Offset-Dumper: Scan signatures and ... https://github.com/guidedhacking/GH-Offset-Dumper

16. Hacking and reverse engineering il2cpp games with ghidra https://noob3xploiter.medium.com/hacking-and-reverse-engineering-il2cpp-games-with-ghidra-5cee894024f2
17. Finding loaders for obfuscated global-metadata.dat files https://katyscode.wordpress.com/2021/02/23/il2cpp-finding-obfuscated-global-metadata/
18. What's this global-metadata.dat thing and why does it matter? https://www.guardingpearsoftware.com/blog/whats-this-global-metadatadat-thing-and-why-does-it-mat-36565
19. PUBG MOBILE Emulator Detection Avoidance Cheat https://pubgmobile.proximabeta.com/webplat/info/news_version3/35372/57705/57706/57821/57822/57823/m22591/201911/836731.shtml
20. Anti-Cheat Expert https://www.tencentcloud.com/product/ace
21. Anti Cheat · gmh5225/awesome-game-security Wiki https://github.com/gmh5225/awesome-game-security/wiki/Anti-Cheat
22. MASTG-KNOW-0031: Emulator Detection https://mas.owasp.org/MASTG-KNOW-0031/
23. atiksoftware/pubg_mobile_memory_hacking_examples: ... https://github.com/atiksoftware/pubg_mobile_memory_hacking_examples
24. Dev Letter: Anti-ESP - NEWS https://pubg.com/en/news/6957
25. Memory read/write library : r/cpp https://www.reddit.com/r/cpp/comments/hpw6az/memory_readwrite_library/
26. Checklist for Optimizing AI Latency with Async Processing https://nano-gpt.com/blog/optimizing-ai-latency-async-processing-checklist
27. HOWTO: Possible Lagless VSYNC for Emulator Devs ... https://www.vogons.org/viewtopic.php?t=95018
28. How To Reduce Lag - A Guide To Better System Latency https://www.nvidia.com/en-us/geforce/guides/gfecnt/202010/system-latency-optimization-guide/
29. Re: Emulator Developers: Lagless VSYNC ON Algorithm https://forums.blurbusters.com/viewtopic.php?t=3972&start=80
30. Rendering latency control, GL.Finish() and VSync - Graphics https://discussions.unity.com/t/rendering-latency-control-gl-finish-and-vsync/924961
31. Rendering issues in Pubg Mobile on Gameloop 7.1 with ... https://www.facebook.com/groups/egcglobal/posts/1162295850917613/
32. PUBG Mobile Season 31 on GameLoop PC https://gameloop.blog/news/pubg-mobile-season-31-best-settings-gameloop-pc/

33. Inter-VM Shared Memory device https://www.qemu.org/docs/master/system/devices/ivshmem.html
34. Shared memory between guest and host in QEMU https://stackoverflow.com/questions/77675263/shared-memory-between-guest-and-host-in-qemu
35. Ashmem (Anonymous Shared Memory) https://aospinsider.com/courses/aosp-course-2/06-ashmem-anonymous-shared-memory/
36. LDPlayer - Lightweight & Fast Android Emulator for PC https://www.ldplayer.net/
37. LDPlayer 14 vs LDPlayer 9 vs BlueStacks: 21% FPS Gap [2026] https://tech-insider.org/ca/ldplayer-14-vs-ldplayer-9-vs-bluestacks-2026/
38. "LDPlayer 14 officially launches, redefining the Android ... https://www.gamespress.com/en-US/LDPlayer-14-officially-launches-redefining-the-Android-gaming-experien
39. Using Frida on Android without root https://koz.io/using-frida-on-android-without-root/
40. 09 - How to use Frida on a non-rooted device - LIEF https://lief.re/doc/stable/tutorials/09_frida_lief.html
41. frida-gadget https://pypi.org/project/frida-gadget/
42. LDPlayer ADB Command Usage Guide (Updated 26/9/3) https://www.ldplayer.net/support/ldplayer-adb-command-usage-guide.html
43. Dumping Android Application Memory: Pentester's Full Guide https://www.redfoxsec.com/blog/dumping-android-application-memory-a-pentesters-complete-guide
44. PUBG Mobile emulator cheats 2026 — guide: ESP, aimbot, ... https://ivsofte.biz/en/blog/pubg-mobile-emulator-cheats-guide/
45. BlueStacks vs LDPlayer vs GameLoop: Android Emulator ... https://tech-insider.org/bluestacks-vs-ldplayer-vs-gameloop-android-14-gap-2026/
46. proc_pid_maps(5) - Linux manual page https://man7.org/linux/man-pages/man5/proc_pid_maps.5.html
47. LDPlayer 14 Setup: 12 Steps, Hyper-V Fix [2026] - Tech Insider https://tech-insider.org/how-to-set-up-ldplayer-14-emulator-2026/
48. sunmughan/pubg_mobile_memory_hacking https://github.com/sunmughan/pubg_mobile_memory_hacking
49. [Request] PUBG 2026 Latest GWorld Offset - May 2026 Patch https://www.unknowncheats.me/forum/other-games/752211-pubg-2026-gworld-offset-2026-patch.html
50. LDPlayer ldconsole Command Line Developer Usage ... https://www.ldplayer.net/support/ldplayer-ldconsole-command-line-developer-usage-guide.html

51. Introduction to LDPlayer Command Line Interface https://www.ldplayer.net/support/introduction-to-ldplayer-command-line-interface.html
52. LDPlayer Command Line Guide | PDF | Computing | Software https://www.scribd.com/document/912875024/CLI-UTILIZATION-IN-ORDER-TO-QUICKLY-USE-BLUE-LD-PLAYER-VIA-THE-COMMAND-LINE-INTERFACE
53. Windows Hypervisor Platform https://www.qemu.org/docs/master/system/whpx.html
54. noahware/hyper-reV: memory introspection and reverse ... https://github.com/noahware/hyper-reV
55. WinVisor – A hypervisor-based emulator for Windows x64 ... https://www.elastic.co/security-labs/blog/winvisor-hypervisor-based-emulator
56. Android SELinux Internals Part 3 - Kernel-Level ... https://www.8ksec.io/android-selinux-internals-part-iii/